Anyway, this is fine as far as it goes, it's a small bit of code that does a small thing. We are somewhat limited though, in that it is awkward to test a code in the same file (not impossible, but generally the Ruby test libraries assume they will be in their own files). We've put a bunch of variables in the global name space, which is not ideal, and generally, this code is not well situated to manage complexity.

You might think that because Bundler requires a separate Gemfile, you can't use it with a standalone single-file Ruby script. Actually, however, you can, through the magic of bundler/inline.

Here's an admittedly contrived example where we want to print the date that we are requesting an Anagram report, and we want to use the date_by_example gem to display the date. By requiring bundler/inline we can then use a method called gemfile that takes a block, like so:

gems/anagram_inline.rb

```
#!/usr/bin/env ruby

require "optparse"

require "bundler/inline"

gemfile do
  source "https://rubygems.org"
  gem "date_by_example"
end

dictionary = "/usr/share/dict/words"

OptionParser.new do |opts|
  opts.banner = "Usage:  anagram [ options ]  word..."

  opts.on("-d", "--dict path", String, "Path to dictionary") do |dict|
    dictionary = dict
  end

  opts.on("-h", "--help", "Show this message") do
    puts opts
    exit
  end

  begin
    ARGV << "-h" if ARGV.empty?
    opts.parse!(ARGV)
  rescue OptionParser::ParseError => e
    warn e.message, "\n", opts
    exit(-1)
  end
end

# convert "wombat" into "abmotw". All anagrams share a signature
def signature_of(word)
  word.unpack("c*").sort.pack("c*")
end

signatures = {}

File.foreach(dictionary) do |line|
  word = line.chomp
  signature = signature_of(word)
```